

Security Assessment Report



Permissioned GSM

November 2026

Prepared for Aave Labs

Table of Contents

Project Summary **3**
- Project Scope 3
- Project Overview 3
- Protocol Overview 3

Assessment Methodology **5**

Threat and Security Overview **6**
- Findings Summary 8
- Severity Matrix 8

Detailed Findings **9**
- Informational Severity Issues 10
  - I-01: Swap roles can only be granted or revoked through governance 10
  - I-02: canSwap() ignores the new permission checks 11
  - I-03: Storage layout mismatch when upgrading GSMs to the permissioned version 12

Disclaimer **13**

About Certora **13**

Project Summary

Project Scope

Project Name	Repository Link	Commit Hash	Platform
Permissioned GSM	https://github.com/aave-dao/gho-origin	f89faa3 (initial) d619fff (fix review)	Solidity

Project Overview

This document describes the security review of **Permissioned GSM**. The work was undertaken from **November 4th, 2026** to **November 6th, 2026**.

The following contract list is included in our scope:

src/contracts/facilitators/gsm/PermissionedGsm.sol
src/contracts/facilitators/gsm/PermissionedGsm4626.sol

The team performed a manual audit of all the files in scope**.**

Protocol Overview

The GHO Stability Module (GSM) exchanges an underlying asset for GHO and back, at a price set by a price strategy, within an exposure cap and with an optional fee. GHO is drawn from and returned to a GhoReserve. The Permissioned GSM adds a restriction on who may swap.

Two contracts implement it:

- `PermissionedGsm` extends `Gsm`.
- `PermissionedGsm4626` extends `Gsm4626`, the variant for ERC4626 vault shares.

Both add the same code. The only changes to the base contracts make `_buyAsset`, `_sellAsset` and `initialize` in `Gsm` overridable.

Two mechanisms define the restriction:

- **Swap roles.** `BUY_ASSET_ROLE` is required to buy the underlying with GHO, and `SELL_ASSET_ROLE` to sell the underlying for GHO. The check is made against the originator of the swap, after the frozen and seized checks and before pricing, hooks and transfers. For a direct call the originator is the caller. For a signed swap it is the signer, and the relayer needs no role. The receiver is never checked.
- **Access flags.** Buys and sells each have a flag, off by default. When the buy flag is on, the `BUY_ASSET_ROLE` check is skipped and any address may buy. The sell flag does the same for sells.

Both contracts override `initialize` to make `SWAP_ACCESS_MANAGER_ROLE` the admin of the two swap roles.

Users:

- **Governance** holds `DEFAULT_ADMIN_ROLE`, which grants and revokes `SWAP_ACCESS_MANAGER_ROLE`.
- **The swap access manager** (`SWAP_ACCESS_MANAGER_ROLE`) grants and revokes both swap roles, and turns the two flags on and off through `setBuyAccessOpen` and `setSellAccessOpen`. `initialize` does not assign this role.
- **Approved counterparties** hold one or both swap roles. Liquidity is shared between them: the module keeps no per-participant balances.
- **The swap freezer and liquidator** are unchanged. A freeze or a seizure blocks swaps for every participant.

Assessment Methodology

Our assessment approach combines design level analysis with a deep review of the implementation to ensure that a protocol is secure, economically sound, and behaves as intended under realistic conditions.

At the design level, we evaluate the architecture, the economic assumptions behind the protocol, and the safety properties that should hold independently of a specific chain or environment. This process includes reviewing internal and cross protocol interactions, state transition flows, trust boundaries, and any mechanism that could be exploited to extract value, deny service, or alter core system behavior. At this stage, a focused threat modelling exercise helps identify key attack surfaces and adversarial capabilities relevant to the system. Design level issues often relate to incentive structures, governance implications, or systemic behavior that emerges under adversarial conditions.

Implementation analysis focuses on the concrete behavior of the code within the execution model of the target chain. This involves reviewing the correctness of logic, access control, state handling, arithmetic behavior, and the nuanced behaviors of the chain environment. Familiar classes of vulnerabilities such as reentrancy conditions, faulty permission checks, precision issues, or unsafe assumptions often surface at this layer. These findings require context aware reasoning that takes into account both the code and the architectural intent.

To support this analysis, the codebase is examined through repeated manual passes and supplemented by automated tools when appropriate. High-risk logic areas receive deeper scrutiny, invariants are validated against both design intent and actual implementation, and potential vulnerability leads are thoroughly investigated. Automated techniques such as static analysis, fuzzing, or symbolic execution may be used to complement manual review and provide additional insight.

Collaboration with the development team plays an important role throughout the audit. This helps confirm expected behaviors, clarify design assumptions, and ensure an accurate understanding of the protocol's intended operation. All findings are documented with clear reasoning, reproducible examples, and actionable recommendations. A follow up review is conducted to validate the applied fixes and verify that no regressions or secondary issues have been introduced.

Threat and Security Overview

Scope and Trust Assumptions

The review focused on logic errors and vulnerabilities in the access restriction added by PermissionedGsm and PermissionedGsm4626, and in how it interacts with the swap, signature and upgrade paths. The review was conducted through manual analysis of properties, invariants and hypotheses, supported by proof-of-concept tests in the repository's Foundry suite and on a mainnet fork.

As security analysis is inherently creative and any line of code may introduce vulnerabilities, the following sections outline the primary categories of risks evaluated rather than an exhaustive list.

The following are treated as trust assumptions:

- **Role holder integrity:** the holders of `DEFAULT_ADMIN_ROLE` and `SWAP_ACCESS_MANAGER_ROLE` are not compromised. `SWAP_ACCESS_MANAGER_ROLE` grants and revokes the swap roles and can lift the restriction on buys or on sells, after which any address may buy or sell. `DEFAULT_ADMIN_ROLE` decides who holds `SWAP_ACCESS_MANAGER_ROLE`.
- **Approved counterparties:** an address granted a swap role is the intended participant. Only the originator is checked, so if a role is granted to a contract, anyone who can make that contract swap can use the module.

Security Considerations and Threat Analysis

Access Control Coverage. Every buy must pass the `BUY_ASSET_ROLE` check and every sell the `SELL_ASSET_ROLE` check, unless the matching flag is on. Threats include:

- A swap entry point that does not route through the overridden `_buyAsset` or `_sellAsset`.
- The role being checked against the relayer or the receiver instead of the originator.
- The buy role or buy flag allowing a sell, or the sell role or sell flag allowing a buy.

Signature-Based Swaps. The role is required of the signer, not of the relayer. Threats include:

- A relayer executing a swap for a signer who holds no role.
- A signed swap that fails the role check keeping its nonce, and executing later if the role is granted back before the deadline.
- A signed swap being replayed on another instance or chain.

Role and Flag Administration. Threats include:

- Revoking a swap role taking a full governance cycle when only governance can do it.
- The buy or sell flag being turned on, which lets any address buy or sell without holding a role.
- `canSwap()` returning true for a caller without the swap role, since it only checks the frozen and seized state.

Upgradeability and Storage. Threats include:

- The two flags colliding with existing `Gsm` state.
- A proxy whose storage layout predates the current `Gsm` being upgraded in place.
- State later added to `Gsm` or `Gsm4626` moving the two flags to a different storage position.

Adversarial Incentives

1. **Unauthorized swapping:** an address without a swap role obtains GHO or the underlying asset through an entry point that skips the check, through a relayer, or through a contract that holds a role.
2. **Evading revocation:** a counterparty whose role is about to be revoked swaps before the revocation executes, or relies on a swap signed before it.

Findings Summary

The table below summarizes the findings of the review, including type and severity details.

Severity	Discovered	Confirmed	Fixed
Critical	-	-	-
High	-	-	-
Medium	-	-	-
Low	-	-	-
Informational	3	3	2
Total	3	3	2

Severity Matrix

Impact	High	Medium	High	Critical
	Medium	Low	Medium	High
	Low	Low	Low	Medium
		Low	Medium	High
		Likelihood		

Detailed Findings

ID	Title	Severity	Status
I-01	Swap roles can only be granted or revoked through governance	Informational	Fixed
I-02	canSwap() ignores the new permission checks	Informational	Fixed
I-03	Storage layout mismatch when upgrading GSMs to the permissioned version	Informational	Acknowledged

Informational Severity Issues

I-01: Swap roles can only be granted or revoked through governance

Files:

src/contracts/facilitators/gsm/PermissionedGsm.sol,
src/contracts/facilitators/gsm/PermissionedGsm4626.sol

Description:

BUY_ASSET_ROLE and SELL_ASSET_ROLE are administered by DEFAULT_ADMIN_ROLE, which is expected to be held by the governance executor, as on the existing GSMs. Revoking a counterparty therefore takes a full proposal cycle, during which the counterparty can keep swapping and can see the revocation coming.

Recommendations:

Allow a trusted multisig to revoke swap roles immediately. Ideally its power is limited to revocation, with granting left to governance.

Customer Response:

`SWAP_ACCESS_MANAGER_ROLE` is the admin of `BUY_ASSET_ROLE` and `SELL_ASSET_ROLE`, set in `initialize`, and gates `setBuyAccessOpen` and `setSellAccessOpen`. The role goes to a Risk Council multisig, or to a steward contract that the Risk Council controls.

Fix Review:

Fixed in d7add81.

I-02: canSwap() ignores the new permission checks

Files:

src/contracts/facilitators/gsm/Gsm.sol

Description:

canSwap() only checks that the GSM is not frozen or seized. In the permissioned GSM a swap also requires the originator to hold BUY_ASSET_ROLE or SELL_ASSET_ROLE, unless that side has been opened. As a result, canSwap() can return true for an address whose swap will revert, and integrations relying on it receive an incorrect answer.

Recommendations:

Add a view that takes the originator and the direction, e.g. canSwap(address originator, bool isBuy), and keep the existing canSwap() as the global check.

Customer Response:

The NatSpec of `canSwap()` now states that it reports whether swaps are operational and ignores the caller's permissions and any applicable limits. A user's authorization can be verified with `hasRole` and the corresponding swap role.

Fix Review:

Documented in d619fff.

I-03: Storage layout mismatch when upgrading GSMs to the permissioned version

Description:

The PR supports upgrading an existing GSM proxy to PermissionedGsm / PermissionedGsm4626. This only works if the proxy's storage layout matches the new code.

PR #41 switched the EIP712 parent of Gsm to the OpenZeppelin v5 version, which adds two storage variables. This moves every `Gsm` variable 2 slots later. If an already deployed GSM that uses the old layout is upgraded, the new code would read the wrong slots and the contract would break. However, the ARFC says the first permissioned GSM is a new deployment on Avalanche and that existing GSMs stay unaffected, so this upgrade is not planned. Future upgrades of the base contract should also be considered. The access flags are stored right after the last Gsm variable, and Gsm has no storage gap. If a variable is added to Gsm later, the flags move and the access settings can change.

Recommendations:

- Document that upgrading a normal GSM to a permissioned one only works for GSMs deployed after PR #41. Older GSMs should be migrated, not upgraded in place.
- Protect the access flags from future changes to `Gsm`, for example with namespaced storage (ERC-7201) or a storage gap at the end of `Gsm`.

Customer Response:

Acknowledged. There are no plans to upgrade older GSMs.

Disclaimer

Even though we hope this information is helpful, we provide no warranty of any kind, explicit or implied. The contents of this report should not be construed as a complete guarantee that the contract is secure in all dimensions. In no event shall Certora or any of its employees be liable for any claim, damages, or other liability, whether in an action of contract, tort, or otherwise, arising from, out of, or in connection with the results reported here.

About Certora

Certora is a Web3 security company that provides industry-leading formal verification tools and smart contract audits. Certora's flagship security product, Certora Prover, is a unique SaaS product that automatically locates even the most rare & hard-to-find bugs on your smart contracts or mathematically proves their absence. The Certora Prover plugs into your standard deployment pipeline. It is helpful for smart contract developers and security researchers during auditing and bug bounties.

Certora also provides services such as auditing, formal verification projects, and incident response.